Các Design Pattern đề xuất sử dụng trong phần mềm xử lý ảnh SAR

Báo cáo sau đây sẽ đề xuất các mẫu thiết kế (Design Pattern) có thể/nên sử dụng trong phần mềm xử lý ảnh SAR. Cấu trúc bản báo cáo sẽ bao gồm 2 phần:

- Phần I sẽ định nghĩa mẫu thiết kế, một số lập luận về vấn đề mẫu thiết kế.
- Phần II sẽ đề cập đến các mẫu thiết kế nên sử dụng. Hầu hết các mẫu thiết kế này đang được phần mềm SNAP áp dụng.

Các thông tin/kiến thức được tham khảo nguồn từ Internet + Mã nguồn của phần mềm SNAP.

I. Mẫu thiết kế

Định nghĩa

> Mẫu thiết kế là một giải pháp thiết kế tổng quát, có thể tái sử dụng, cho các vấn đề thường gặp trong thiết kế phần mềm.

Các lợi ích của dùng mẫu thiết kế trong chương trình có thể liệt kê như sau:

- Giúp chuẩn hoá cách thiết kế
- Tăng khả năng tái sử dụng kiến trúc
- Giảm phụ thuộc giữa các module
- Dễ dàng mở rộng và bảo trì: Mã nguồn viết nên có tính đóng để tránh sửa đổi mã cũ, nhưng cũng phải có tính mở để sẵn sàng mở rộng tính năng (Nguyên tắc SOLID, hầu hết các mẫu thiết kế được xây dựng dựa trên nguyên tắc SOLID).
- Có một ngôn ngữ chung giữa các developer: Thay vì phải giải thích "Tôi muốn áp dụng một kiến trúc để thanh process bar hiện thị đúng tiến độ xử lý cho quá trình lọc nhiễu ảnh của tôi, đồng thời khi hoàn thành xong tôi muốn có một thông báo hiển thị cho người dùng", ta có thể "Tôi sẽ áp dụng Observer Pattern để thiết kế giao diện dựa trên quy trình lọc nhiễu ảnh của tôi".

Nhưng

Mẫu thiết kế sinh ra để giải quyết một vấn đề cụ thể tồn tại trong chương trình. Tuy nhiên, nếu áp dụng mẫu thiết kế một cách bừa bãi sẽ dẫn đến khiến cho chương trình trở nên phức tạp không đáng có, hay gọi là "thiết kế trước tương lai" (permature abstraction).

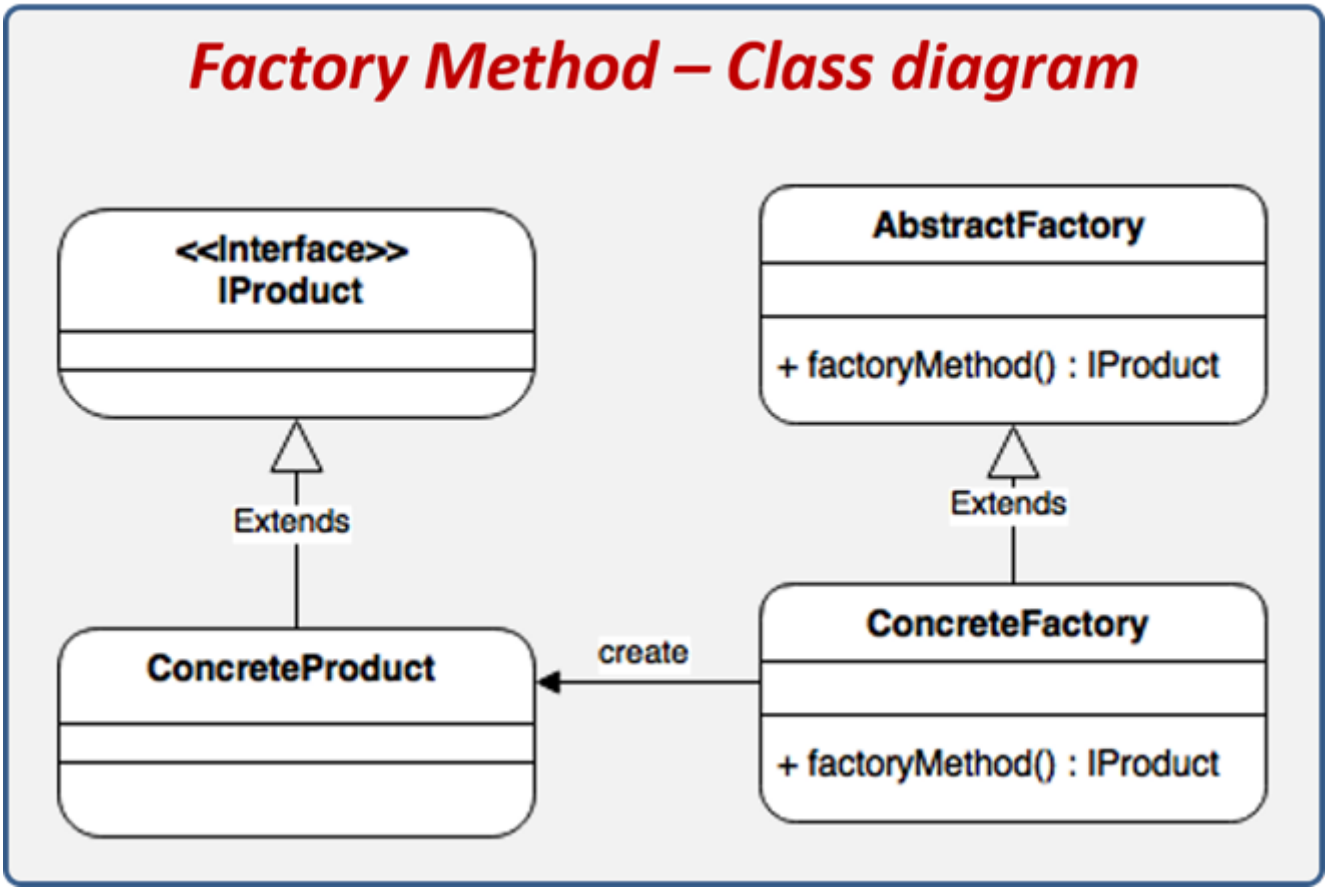
> Premature optimization is the root of all evil.

Ví dụ, chương trình của ta hỗ trợ 2 bộ lọc nhiễu lốm đốm `AFilter` và `BFilter`, có thể trong tương lai ta có thể thêm một vài bộ lọc khác. Giả sử giờ ta muốn thêm bộ lọc `CFilter`, ta có thể sẽ phải quay về sửa mã nguồn cũ, và ta có thể đã vi phạm Open/Closed Principle trong nguyên tắc SOLID:

```
// Một đoạn code được trích trong chương trình SNAP.
  private SpeckleFilter createFilter() {
```

```
        switch (filter) {
            case BOXCAR_SPECKLE_FILTER:
                return new Boxcar(this, sourceProduct, targetProduct, filterSizeX,
filterSizeY,
                        targetBandNameToSourceBandName);

            case MEDIAN_SPECKLE_FILTER:
                return new Median(this, sourceProduct, targetProduct, filterSizeX,
filterSizeY,
                        targetBandNameToSourceBandName);
            // Mỗi lần ta muốn thêm bộ lọc mới, ta phải cập nhập code cũ ở đây
            default:
                return null;
        }
    }
```

Theo như lý thuyết, ta có thể sử dụng mẫu thiết kế "Factory Method":



Factory method sẽ giúp ta có thể dễ dàng mở rộng các bộ lọc mới mà không cần thay đổi mã nguồn cũ. Tuy nhiên, mã nguồn sẽ trở nên phức tạp hơn khá nhiều bằng cách tạo thêm các class mới:

```
// Interface filter
class Filter {
public:
    virtual void doSth() const = 0;
    virtual ~Filter() = default;
```

```
};

class LeeFilter : public Filter {
public:
    void doSth() const override{
        std::cout << "Do LeeFilter techniques"<<std::endl;
    }
};

class MedianFilter : public Filter{
public:
    void doSth() const override {
        std::cout << "Do MedianFilter techniques"<<std::endl;
    }
};

// IFactory
class FilterFactory {
public:
    virtual std::unique_ptr<Filter> create() const = 0;
    virtual ~FilterFactory() = default;
};

// Concrete Factory
class LeeFilterFactory : public FilterFactory {
public:
    std::unique_ptr<Filter> create() const override {
        return std::make_unique<LeeFilter>();
    }
};

class MedianFilterFactory : public FilterFactory {
public:
    std::unique_ptr<Filter> create() const override {
        return std::make_unique<MedianFilter>();
    }
};



int main() {
    // User choose "Lee Filter Button" --> Program call "LeeFilterFactory"
    LeeFilterFactory b;
    auto a = b.create();
    return 0;

}
```

Vậy nên ta nên nắm được, mẫu thiết kế không phải là **mục tiêu**, nó chỉ là **công cụ** để giải quyết một vấn đề. Nhiều người sẽ học các mẫu thiết kế rồi, cố "Chỗ này nhét mẫu thiết kế vào được không nhỉ?". Đấy là cách sai. Thực tế ta nên:

1. Viết mã đơn giản trước
2. Khi thấy mã nguồn bắt đầu:
    - Lặp lại
    - Khó mở rộng
    - Khó thay đổi
3. Khi đấy có thể áp dụng mẫu thiết kế.

Các loại mẫu thiết kế

Design Pattern được chia làm 3 nhóm chính:

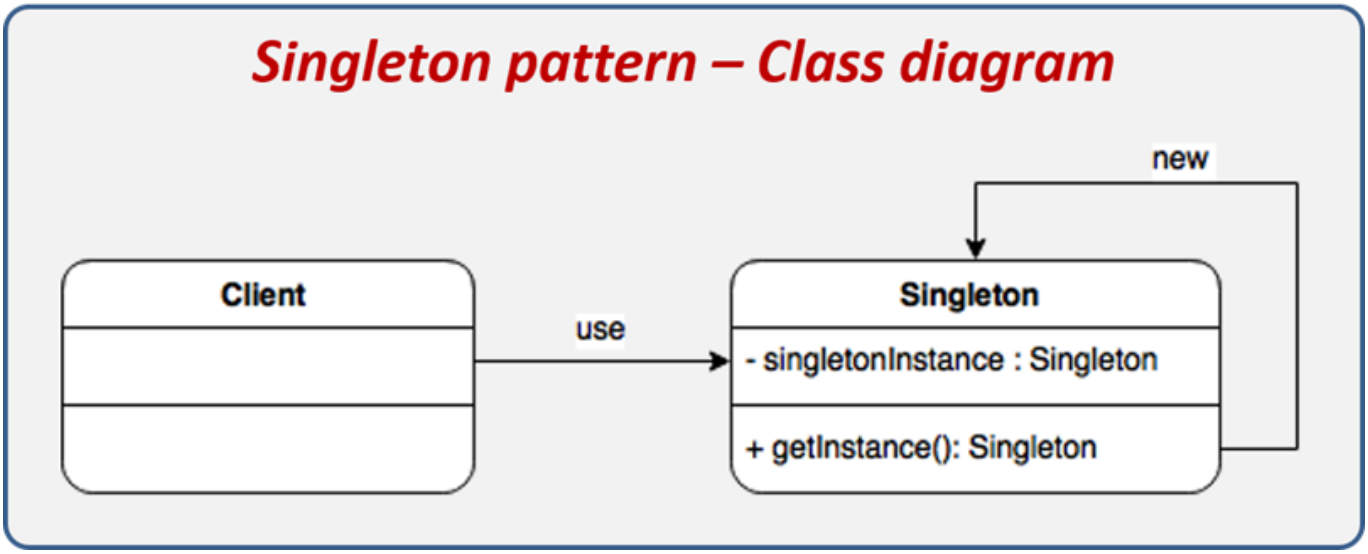
- Creational Patterns: Quản lý cách tạo đối tượng một cách linh hoạt và kiểm soát tốt hơn, tách quá trình tạo object khỏi phần sử dụng object
- Structural Patterns: Tổ chức và kết hợp các lớp/đối tượng để tạo cấu trúc lớn hơn, tập trung vào các cách đối tượng liên kết với nhau.
- Behavioral Patterns: Quản lý cách đối tượng giao tiếp và tương tác với nhau, tập trung vào hành vi và luồng xử lý.

Creational	Structural	Behavioral
Factory	Adapter	Interpreter
Abstract Factory	Bridge	Template Method
Builder	Composite	Chain of Responsibility
Prototype	Decorator	Command
Singleton	Facade	Iterator
	Flyweight	Mediator
	Proxy	Memento
		Observer
		State
		Strategy
		Visitor

Các mẫu thiết kế có thể sẽ được sử dụng (Đang được SNAP áp dụng)

Singleton Pattern (Creational Pattern)

Mẫu thiết kế "Singleton Pattern" về bản chất chỉ là đảm bảo class chỉ có một đối tượng tồn tại xuyên suốt trong chương trình. Mẫu Singleton cung cấp hàm truy cập toàn cục `getInstance()` cho những nơi cần truy cập vào đối tượng.



Ứng dụng dễ thấy nhất của mẫu Singleton mà ta có thể áp dụng trong chương trình đó là quản lý Log. Thông thường ta sẽ lưu log và duy nhất một tệp, vì vậy Singleton Pattern có thể được áp dụng nhằm đảm bảo các log ở mọi nơi trong chương trình sẽ được ghi vào đúng một tệp (1 đối tượng duy nhất).

Dựa trên mã nguồn chương trình SNAP, ta có thể quan sát được khá nhiều nơi trong mã nguồn áp dụng Singleton Pattern, đặc điểm nhận dạng mã sẽ gọi `getInstance()` để truy cập đối tượng class đó, đồng thời class được truy cập có chứa biến static private:

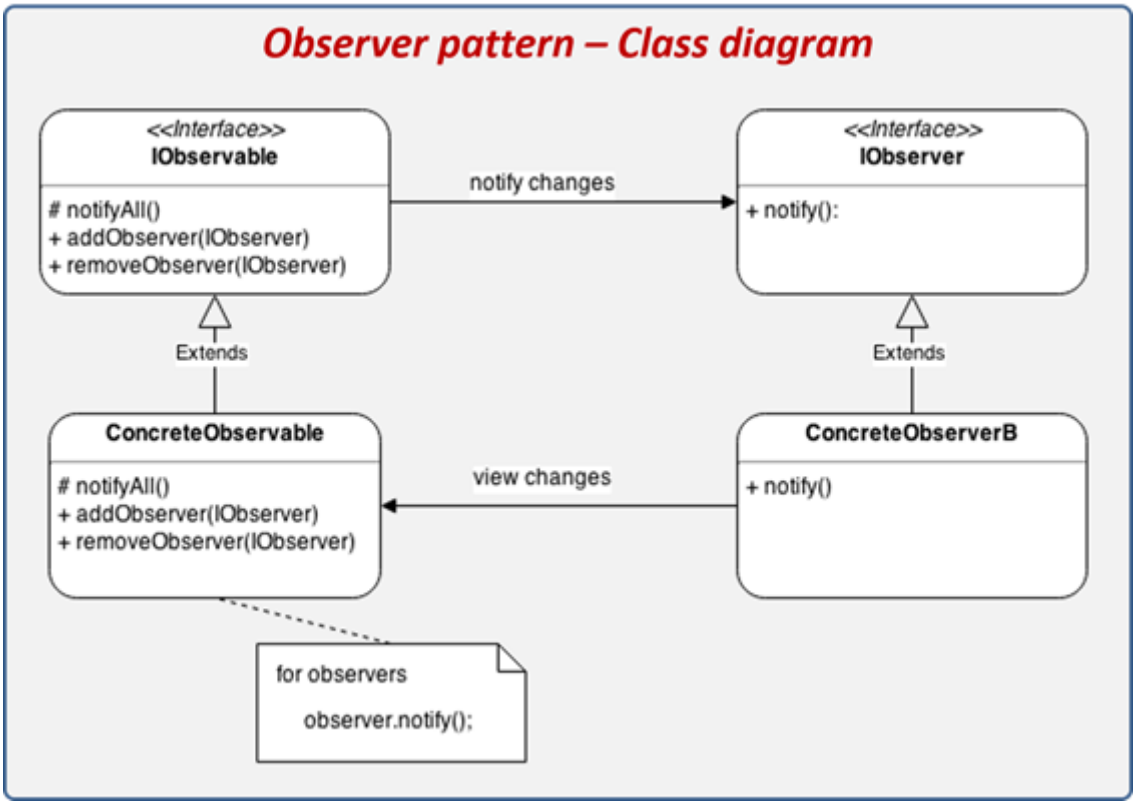
```
    // ví dụ 1: GDALLoader.java
public final class GDALLoader {

    private static final GDALLoader INSTANCE = new GDALLoader();

    private static final Logger logger =
Logger.getLogger(GDALLoader.class.getName());

    private GDALVersion gdalVersion;
    private NativeLibraryClassLoader gdalVersionLoader;

    private Map<Integer, Integer> bandToGDALDataTypes;
    private static Boolean GDALDistributionInitializedOnce = false;

    private GDALLoader() {
    }

    /**
     * Returns instance of this class.
     *
     * @return the instance of this class.
     */
    public static GDALLoader getInstance() {
        return INSTANCE;
    }
    ...
}
    // ví dụ 2:
    public File retrieveOrbitFile(final String orbitType) throws Exception {
```

```
    dorisReader = EnvisatOrbitReader.getInstance();
    final int absOrbit = absRoot.getAttributeInt(AbstractMetadata.ABS_ORBIT, 0);
    }
    //... Nhiều nơi khác
```

Mã nguồn ví dụ sử dụng Singleton Pattern: 1

Observer Pattern (Behavioral Pattern)

Có thể giải thích Observer Pattern hoạt động như việc đăng kí theo dõi một youtuber vậy, nếu chúng ta bấm chọn đăng kí youtuber đó, chúng ta sẽ trở thành những người theo dõi, và mỗi lần youtuber đó đăng một video mới, ta sẽ nhận được thông báo. Observer Pattern hoạt động tương tự như vậy với cơ chế observer/listener.



SNAP sử dụng rộng rãi Observer Pattern nhằm tách phần UI/logic và truyền thông báo mỗi khi có thay đổi. Đặc điểm nhận biết là các lớp áp dụng thường là lớp mở rộng từ lớp Observable và thường gọi notifyObservers() mỗi khi có một thay đổi muốn thông báo cho Listener. ví dụ:

```
// File Project.java
public class Project extends Observable {

    private final static Project _instance = new Project();
    private final List<Listener> listeners = new ArrayList<>();
    ...

    void notifyEvent(boolean saveProject) {
        setChanged();
        notifyObservers();
        clearChanged();
```

```
        if (saveProject)
            SaveProject();

        for (Listener listener : listeners) {
            listener.projectChanged();
        }
    }
```

Comment: Observable là một library trên java, để sử dụng Observer Pattern trên C++, ta có thể sẽ cần tự viết lại class Observer/Observable hoặc là có thể dùng những thư viện có sẵn: Qt signal/slot, boost signals2

Mã nguồn ví dụ sử dụng Observer Pattern: 2

Command Pattern (Behavioral Pattern)

Command Pattern về bản chất có thể hiểu nó sẽ là cấu trúc thực hiện đóng gói các thao tác tương tác thành các đối tượng, có lưu trạng thái đối tượng đó, từ đó có thể thực hiện các thao tác như undo/redo.

Trong SNAP, Command Pattern được hỗ trợ bởi thư viện framework NetBeans ActionID.

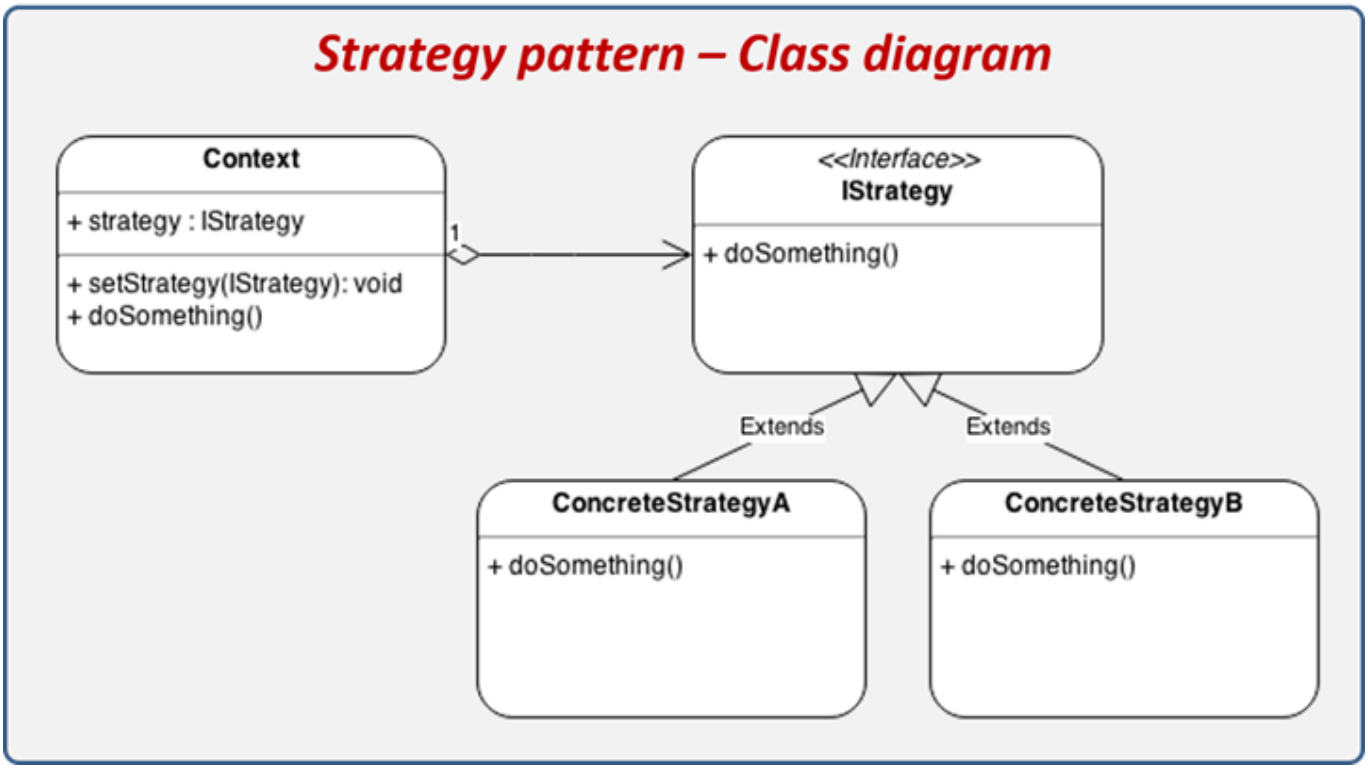
```
public final class OpenProductAction extends AbstractAction {

    // … state (file list, format, prefs) stored in the action object …

    @Override
    public void actionPerformed(ActionEvent e) {
        execute();        // the “command” execution
    }

    public Boolean execute() {
        ProductOpener opener = new ProductOpener();
        opener.setFiles(getFiles());
        // … configure opener …
        return opener.openProduct();
    }
}
```

Comment: Trong Qt, có hỗ trợ QAction để lập trình Command Pattern với C++ Mã nguồn ví dụ sử dụng Command Pattern: 3

Strategy Pattern (Behavioral Pattern)

Strategy Pattern định nghĩa một họ các thuật toán (hoặc hành vi), đóng gói mỗi thuật toán vào một lớp riêng và làm cho chúng có thể hoán đổi cho nhau trong run-time.

Nhìn chung, Strategy Pattern được sử dụng khi bạn muốn thay đổi thuật toán dựa trên các điều kiện trong run-time.



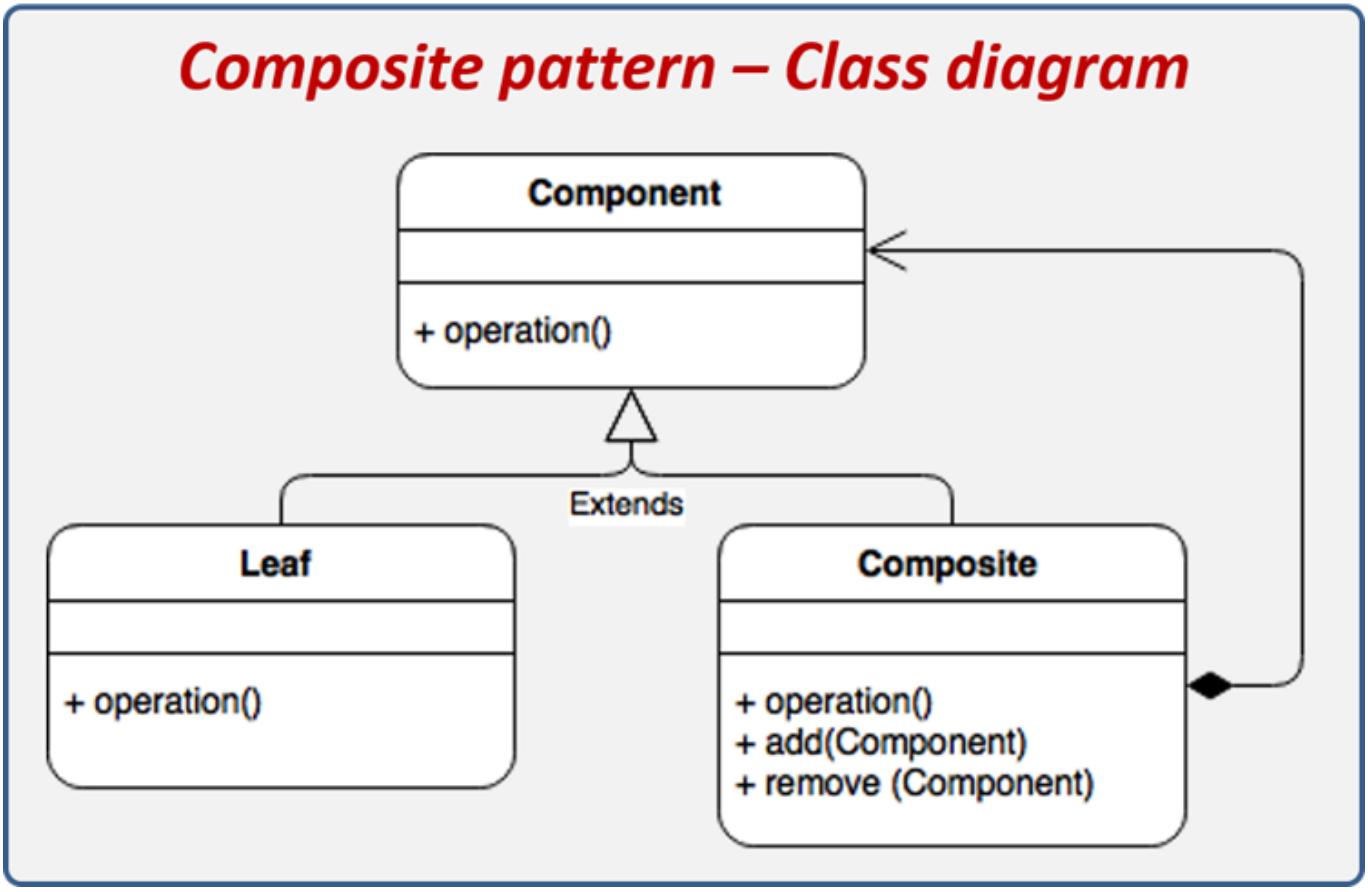
Ví dụ trong SNAP:

```
// trong TemporaryMODISImage.java
private interface Strategy {
    GeoPos getGeoPos(int x, int y);
    int xOffset(int x);
    int yOffset(int y);
    boolean isSourceInvalid(int x, int y);
}

private class SouthStrategy implements Strategy { … }

private class NorthStrategy implements Strategy { … }

…
    if (mode == ModisMosaicer.NORTH_MODE) {
        strategy = new NorthStrategy();
    } else {
        strategy = new SouthStrategy();
    }
    setTileCache(JAI.createTileCache(50L * 1024 * 1024));
…
if (strategy.isSourceInvalid(x,y)) …
GeoPos gp = strategy.getGeoPos(x+xOffset,y+yOffset);
```

Mã nguồn ví dụ sử dụng Strategy Pattern: 4

Composite Pattern (Structural Pattern)

Composite Pattern là một cấu trúc cây nơi một đối tượng (Component) có thể chứa các đối tượng khác (Leaf hoặc Composite) và tất cả đều được xử lý đồng nhất.



Trong SNAP:

```
// cơ sở
public abstract class ProductNode extends ExtensibleObject { … }

// lớp chứa con (composite)
public class ProductNodeGroup<T extends ProductNode> extends ProductNode {
    private final ProductNodeList<T> nodeList = new ProductNodeList<>();

    public void add(T node) { … }
    public void remove(T node) { … }
    public T[] getNodes() { … }
    // …
}

// nhiều lớp con cụ thể
public class Product extends ProductNode implements Closeable { … }
public class Band extends ProductNode { … }
public class TiePointGrid extends ProductNode { … }
… // và nhiều loại node khác
```

Mã nguồn ví dụ sử dụng Composite Pattern: 5

Kết luận

Trên đây là một số Design Pattern được sử dụng nhiều trong SNAP và được đề xuất trong chương trình xử lý ảnh SAR.

Chung quy lại, việc biết và nắm được khái niệm các Design Pattern cũng khá quan trọng khi nó giúp chúng ta biết được các giải pháp khi một vấn đề trong chương trình xuất hiện.